

Code Documentation

Scalable Music Genre Classification and Audio Analytics Using the Free Music Archive Dataset

Submission Artefact Explaining the Notebooks, Scripts, Outputs, and Demo Code

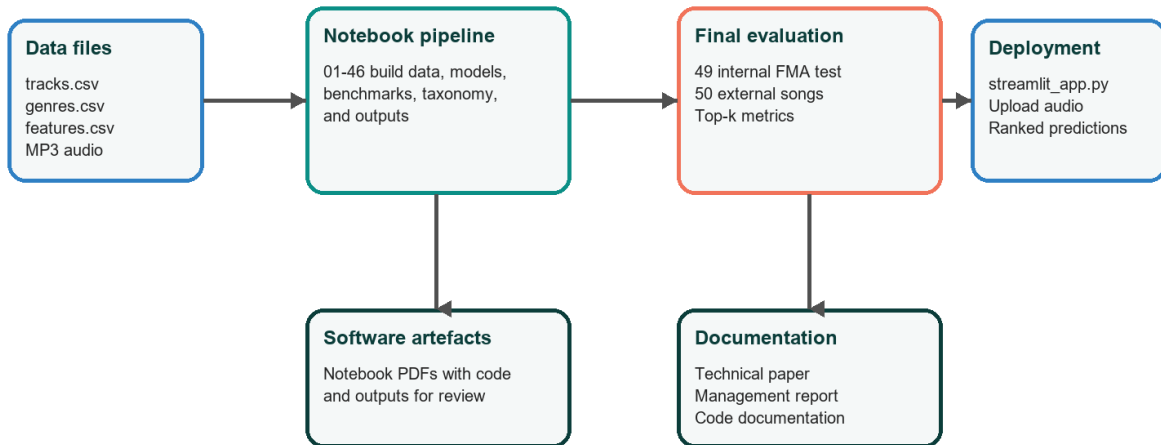
Student: Julian Devonish
 Programme: MSc Applied Data Science
 Course: COMP6830 - Data Science Capstone II
 Supervisor: Dr. Sean Miller
 Prepared: August 9, 2026

Purpose of This Code Documentation

This document explains the software artefacts submitted with the capstone project. It is intended to help a reviewer understand what each notebook, script, output folder, and deployment file does without having to open every file first.

The main source code is contained in the Jupyter notebooks under notebook/. Executed notebook PDFs with code and visible outputs are stored under software artefacts/notebook_pdfs/. The Streamlit demo is implemented in streamlit_app.py and uses the trained model artefacts and metadata files kept in the project folder.

Code Artefact Architecture



The codebase is organised so the notebooks explain the research path, while Streamlit demonstrates the final ranked prediction workflow.

Figure 1. Code artefact architecture and submission relationship.

Figure 1 shows how the codebase moves from data files to notebooks, final evaluation, deployment, and documentation. The notebooks record the research process. The Streamlit app demonstrates the final inference workflow in a user-facing form.

Repository Structure

Folder or file	Role	Review notes
notebook/	Main Jupyter notebook source code from data understanding through final evaluation.	Primary coding artefact. Read alongside notebook_pdfs for executed outputs.
software artefacts/notebook_pdfs/	PDF exports of notebooks with code and outputs.	Useful for marking because code and output are visible without running the notebooks.
software artefacts/executed_notebooks/	Executed copies for notebooks 49 and 50.	Preserves final internal and external evaluation runs.
outputs/	CSV, JSON, and summary files produced by final notebooks.	Contains final Notebook 49 and Notebook 50 evidence files.
models/	Saved trained model artefacts and supporting files.	Used by inference and deployment workflows.
data/	Data manifests, external audio manifest, and project data references.	Large source audio may be excluded from Git where appropriate.
streamlit_app.py	Upload-based demo for ranked genre prediction.	Uses 15-second audio windows and returns Top-k suggestions.
documentation/	Technical paper, management report, deployment notes, data dictionary, and this code documentation.	Main submission documentation location.

Notebook Development Phases

Notebook range	Phase	Count	First artefact	Last artefact
01-06	Foundation and structured modelling	6	01_metadata_understanding.ipynb	06_large_subset_analysis_and_structured_model.ipynb
07-11	Initial audio, hybrid, and prediction pipeline	5	07_audio_model.ipynb	11_final_prediction_pipeline.ipynb
12-22	Expanded and 10-genre experimentation	11	12_expand_8genre_pilot_and_retrain.ipynb	22_final_results_summary_and_comparison.ipynb
23-38	Candidate-150 multi-label development	16	23_full_genre_inventory_and_multilabel_prep.ipynb	38_final_candidate150_consolidation_and_report_tables.ipynb
39-46	Full 163-genre strategy and rare-tail routing	8	39_full161_rare_tail_preparation_and_all_genre_strategy.ipynb	46_final_full_project_consolidation.ipynb
47-48	Final inference pipeline preparation	3	47_final_audio_inference_pipeline.ipynb	48_looped_audio_genre_prediction_with_confidence.ipynb
49	Final internal FMA test evaluation	1	49_final_fma_test_split_batch_evaluation.ipynb	49_final_fma_test_split_batch_evaluation.ipynb
50	External song batch evaluation	1	50_external_song_batch_windowed_inference.ipynb	50_external_song_batch_windowed_inference.ipynb

The notebook sequence is progressive. Early notebooks test data access and structured baselines. Later notebooks expand the label set, add audio CNN modelling, test hybrid fusion, and finish with internal and external evaluation. This sequence is useful because it shows the decisions that led to the final ranked prediction system.

Core Modelling Code

Structured modelling branch

Structured notebooks use FMA metadata and engineered audio features from `features.csv`. These experiments provide interpretable baselines and allow the project to compare tabular learning against audio-content learning.

Audio CNN branch

Audio notebooks convert raw MP3 clips into Mel-spectrogram inputs. The CNN branch became the strongest source of genre signal because it learns from sound patterns such as rhythm, timbre, and production style.

Hybrid fusion branch

Hybrid notebooks combine structured and audio prediction scores. The final benchmark direction used a higher audio weight because the audio branch carried stronger predictive signal in the final experiments.

Rare-tail and full taxonomy handling

The full taxonomy uses 163 genre records: 150 candidate labels predicted directly, 10 rare-tail labels treated through fallback logic, and 3 inventory-only labels retained for taxonomy completeness.

Final Evaluation Notebooks

Notebook	Purpose	Key outputs	Main metrics
49_final_fma_test_split_batch_evaluation.ipynb	Formal internal FMA test-split evaluation using selected test records.	fma_test_summary.json, fma_test_topk_predictions.csv, fma_test_metrics_summary.csv	100 successful tracks; Top-1 72.0%; Top-3 98.0%; Top-5 99.0%; Micro F1 0.450; Macro F1 0.052
50_external_song_batch_windowed_inference.ipynb	External open-licence song batch evaluation using full-song window sampling.	notebook50_external_batch_summary.json, song summary, manifest evaluation, window predictions	60 songs; 8 windows maximum; Top-1 21.7%; Top-3 66.7%; Top-5 83.3%; low-confidence rate 48.3%

These two notebooks should be treated as the final evidence notebooks. Notebook 49 shows performance within the FMA test split. Notebook 50 shows the harder external-audio case. The difference between them is important because it defines the model as a ranked review tool rather than an automatic final genre tagger.

MongoDB Implementation Note

MongoDB was implemented in Notebook 04 and expanded in Notebook 25 as the metadata and label-management layer for the project. The local `fma_capstone` database contained `genres_lookup` with 163 genre records and `tracks_multilabel` with 81,574 track documents.

The later modelling and Streamlit app did not query MongoDB live because repeated model training and audio inference were already constrained by local CPU, memory, and disk I/O. Exported CSV, JSON, NumPy, and model files were used for stable reruns and easier submission review. In a production version, MongoDB would be used to retrieve track metadata, maintain the genre hierarchy, store candidate and rare-tail flags, and save prediction results for reviewer search and audit.

Streamlit Demo Code

The deployment demonstration is contained in `streamlit_app.py`. It loads the trained model and metadata, accepts an uploaded audio file, samples windows from the song, converts each window into a Mel-spectrogram input, and returns ranked genre predictions.

Function or component	Purpose	Reviewer interpretation
<code>load_model()</code> , <code>load_metadata()</code>	Load saved model and genre metadata needed by the app.	Separates deployment loading from prediction logic.
<code>window_starts()</code> , <code>extract_window()</code>	Select and extract 15-second windows from uploaded audio.	Allows full songs to be represented by multiple segments.
<code>build_mel_input()</code>	Convert audio samples into model-ready Mel-spectrogram tensors.	Matches the audio representation used by the CNN branch.
<code>predict_song()</code>	Aggregate window predictions and produce Top-1, Top-3, Top-5, scores, confidence, and window votes.	This is the main inference function used by the demo.
<code>confidence_label()</code>	Translate score ranges into readable confidence labels.	Helps non-technical reviewers interpret uncertainty.
<code>show_project_results()</code> , <code>main()</code>	Render the Streamlit interface, results area, and explanatory text.	Connects the technical model to a presentation-ready demo.

Important Output Fields and Metrics

Field or metric	Meaning	Where used
Top-1	The highest-ranked predicted genre.	Demo, Notebook 49, Notebook 50
Top-3 / Top-5	Whether the accepted or expected genre appears within the first three or five ranked suggestions.	Main evaluation framing
<code>main_score</code>	Model score for the main predicted genre.	External song summary and demo output
confidence label	Readable confidence category based on the main score.	Streamlit and Notebook 50 output
window votes	Number of 15-second windows supporting a genre in Top-1, Top-3, or above threshold.	Full-song interpretation
<code>low_confidence</code>	Flag showing that the main score is below the confidence threshold.	Risk-aware reporting
Micro F1	Multi-label F1 weighted toward common labels.	Notebook 49 metrics
Macro F1	Average F1 across labels, sensitive to rare-label weakness.	Notebook 49 metrics

How to Review or Re-run the Code

Use the notebook PDFs first when assessing the submitted code. They preserve code cells and outputs and avoid the need to rerun long experiments.

Use the `.ipynb` files when code inspection or rerunning is required. The notebooks are numbered to show the execution and research order.

Use `outputs/notebook49_fma_test_batch_evaluation/` for final internal test evidence and `outputs/notebook50_external_song_batch_windowed/` for final external evaluation evidence.

Use `streamlit_app.py` for the deployed demo logic. The demo should be interpreted through Top-k ranking, confidence, and window votes rather than only the first predicted label.

Appendix A: Notebook Register

No.	Notebook file	Phase	Markdown cells	Code cells	Outputs
01	<code>01_metadata_understanding.ipynb</code>	Foundation and structured modelling	0	21	17
02	<code>02_structured_model_features.ipynb</code>	Foundation and	2	18	14

		structured modelling			
03	03_echonest_experiment.ipynb	Foundation and structured modelling	0	10	5
04	04_mongodb_pipeline.ipynb	Foundation and structured modelling	1	13	11
05	05_structured_model_medium.ipynb	Foundation and structured modelling	1	15	20
06	06_large_subset_analysis_and_structured_model.ipynb	Foundation and structured modelling	1	22	22
07	07_audio_model.ipynb	Initial audio, hybrid, and prediction pipeline	1	25	27
08	08_improved_audio_model_from_large.ipynb	Initial audio, hybrid, and prediction pipeline	1	15	18
09	09_structured_model_for_audio_hybrid.ipynb	Initial audio, hybrid, and prediction pipeline	1	20	26
10	10_hybrid_fusion_model.ipynb	Initial audio, hybrid, and prediction pipeline	1	13	13
11	11_final_prediction_pipeline.ipynb	Initial audio, hybrid, and prediction pipeline	1	9	5
12	12_expand_8genre_pilot_and_retrain.ipynb	Expanded and 10-genre experimentation	1	11	13
13	13_audio_cnn_expanded.ipynb	Expanded and 10-genre experimentation	1	17	22
14	14_structured_model_expanded.ipynb	Expanded and 10-genre experimentation	1	19	24
15	15_hybrid_fusion_expanded.ipynb	Expanded and 10-genre experimentation	1	13	13
16	16_build_10genre_pilot.ipynb	Expanded and 10-genre experimentation	1	11	13
17	17_audio_cnn_10genre.ipynb	Expanded and 10-genre experimentation	1	18	22
18	18_structured_model_10genre.ipynb	Expanded and 10-genre experimentation	1	20	23
19	19_hybrid_fusion_10genre.ipynb	Expanded and 10-genre experimentation	1	14	15
20	20_final_prediction_pipeline_10genre.ipynb	Expanded and 10-genre experimentation	1	9	5
21	21_batch_inference_demo_10genre.ipynb	Expanded and 10-genre experimentation	1	9	14
22	22_final_results_summary_and_comparison.ipynb	Expanded and 10-genre experimentation	1	9	13
23	23_full_genre_inventory_and_multilabel_prep.ipynb	Candidate-150 multi-label development	1	13	17
24	24_multilabel_target_matrix_and_split_audit.ipynb	Candidate-150 multi-label development	1	14	24
25	25_mongodb_ingestion_for_multilabel_phase.ipynb	Candidate-150 multi-label development	1	14	16
26	26_structured_multilabel_baseline_candidate150.ipynb	Candidate-150 multi-label development	1	19	22
27	27_structured_multilabel_threshold_tuning_candidate150.ipynb	Candidate-150 multi-label development	1	14	15
28	28_structured_multilabel_per_label_thresholds_and_topk_decoding.ipynb	Candidate-150 multi-label development	1	17	18
29	29_structured_multilabel_final_upgrade_candidate150.ipynb	Candidate-150 multi-label development	1	22	31
30	30_audio_multilabel_baseline_candidate150_pilot.ipynb	Candidate-150 multi-label development	1	25	32
31	31_structured_predictions_for_audio_pilot_candidate150.ipynb	Candidate-150 multi-label development	1	16	18
32	32_hybrid_multilabel_fusion_candidate150_pilot.ipynb	Candidate-150 multi-label development	1	14	17
33	33_hybrid_multilabel_per_label_thresholds_candidate150_pilot.ipynb	Candidate-150 multi-label development	1	16	17
34	34_audio_multilabel_candidate150_expanded_pilot.ipynb	Candidate-150 multi-label development	1	25	32
35	35_structured_predictions_for_expanded_audio_candidate150.ipynb	Candidate-150 multi-label development	1	16	17
36	36_hybrid_multilabel_fusion_candidate150_expanded.ipynb	Candidate-150 multi-label development	1	15	18
37	37_expanded_hybrid_multilabel_per_label_thresholds_and_cap.ipynb	Candidate-150 multi-label development	1	16	17
38	38_final_candidate150_consolidation_and_report_tables.ipynb	Candidate-150 multi-label development	1	16	33
39	39_full161_rare_tail_preparation_and_all_genre_strategy.ipynb	Full 163-genre	1	14	25

		strategy and rare-tail routing			
40	40_full161_rare_tail_fallback_router.ipynb	Full 163-genre strategy and rare-tail routing	1	14	25
41	41_full161_end_to_end_inference_pipeline.ipynb	Full 163-genre strategy and rare-tail routing	1	18	24
42	42_full161_single_file_inference_demo.ipynb	Full 163-genre strategy and rare-tail routing	1	17	26
43	43_full161_batch_inference_demo.ipynb	Full 163-genre strategy and rare-tail routing	1	10	13
44	44_full161_targeted_rare_tail_batch_demo.ipynb	Full 163-genre strategy and rare-tail routing	1	12	19
45	45_full161_rare_tail_prototype_reranker.ipynb	Full 163-genre strategy and rare-tail routing	1	18	26
46	46_final_full_project_consolidation.ipynb	Full 163-genre strategy and rare-tail routing	1	14	22
47	47_final_audio_inference_pipeline.ipynb	Final inference pipeline preparation	1	16	23
47	47_final_audio_inference_pipeline_patched_multwindow_safe_stage2.ipynb	Final inference pipeline preparation	1	16	28
48	48_looped_audio_genre_prediction_with_confidence.ipynb	Final inference pipeline preparation	1	19	30
49	49_final_fma_test_split_batch_evaluation.ipynb	Final internal FMA test evaluation	2	13	0
50	50_external_song_batch_windowed_inference.ipynb	External song batch evaluation	2	10	0

The register records all notebook code artefacts in the repository. Some notebooks are exploratory by design, while notebooks 49 and 50 provide the final evaluation outputs used in the technical and management reports.